实验五

一、 实验目的和环境

实验目的：

1. 理解流水线 CPU 的基本原理和组织结构
2. 掌握五级流水线的工作过程和设计方法
3. 理解流水线取指、译码、执行、存储器访问、写回、停机的设计原理和解决方法
4. 设计流水线测试程序

实验环境：

• 操作系统：Windows 11 • 开发工具：Xilinx VIVADO 2023.2 • NEXYS A7 开发板

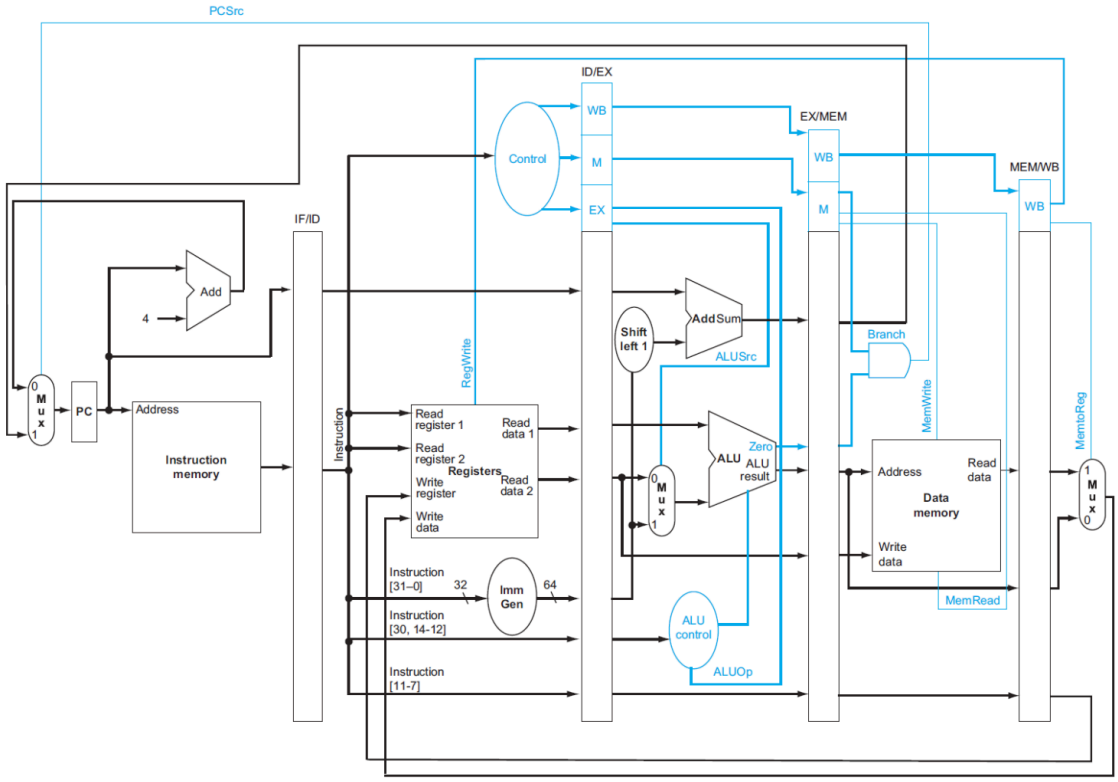
二、 实验内容和原理

2.1. 流水线处理器集成

2.1.1. 实验目标及任务

· 利用五级流水线各级封装模块集成 CPU · 替换 Exp04 的单周期 CPU 为本实验集成的五级流水线 CPU

2.1.2. 实验原理



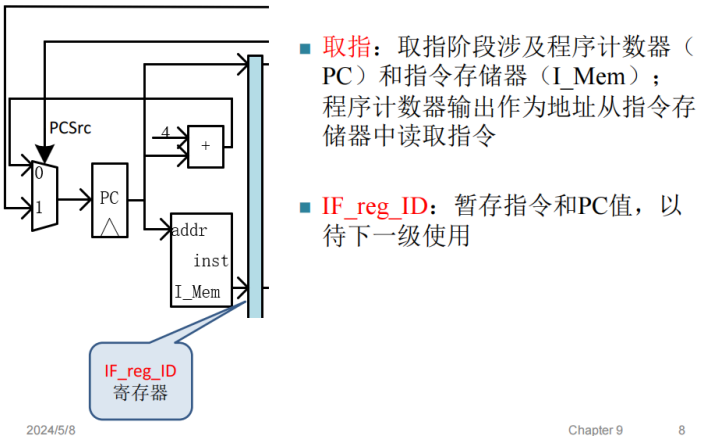
2.2. IF-ID 设计

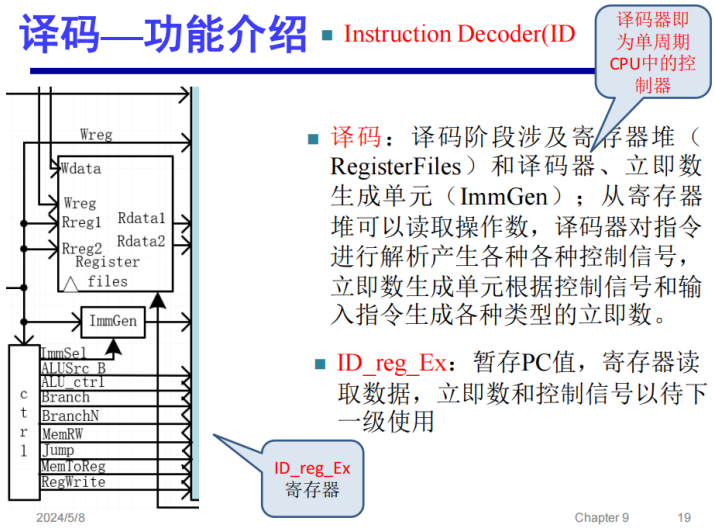
2.2.1. 实验目标及任务

· 熟悉 RISC-V 五级流水线的工作特点，了解取指、译码的原理，掌握 IP 核的使用方法，集成并测试 CPU

· 设计取指（IF）、译码（ID）模块以及相应的寄存器模块

2.2.2. 实验原理





2.3. Ex-Mem-WB 设计与流水线 CPU 集成

2.3.1. 实验目标及任务

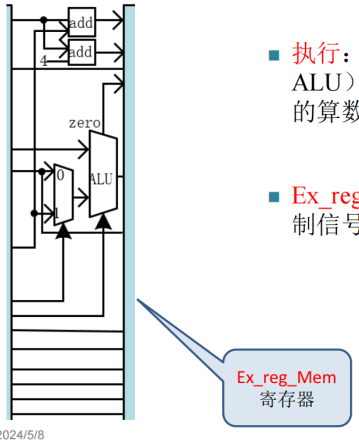
任务一：设计执行（Ex）、存储器访问（Mem）、写回（WB）模块以及对应的流水线寄存器

任务二：流水线 CPU 的集成及测试

· 将 lab5_1 所设计的模块与本实验设计的模块进行集成 · 对集成的流水线 CPU 进行仿真测试 · 对替换流水线 CPU 后的系统进行上板测试

2.3.2. 实验原理

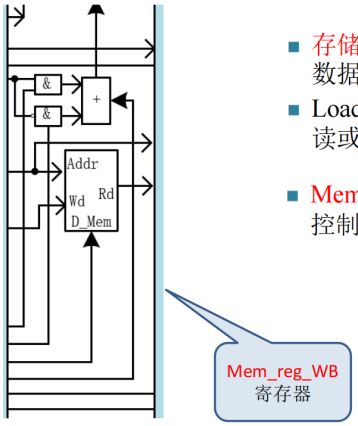
执行—功能介绍 Execute(Ex)



- 执行：执行阶段涉及运算单元（ALU）它获取操作数并完成指定的算数运算或逻辑运算。
- Ex_reg_Mem：暂存运算结果和控制信号，以待下一级使用

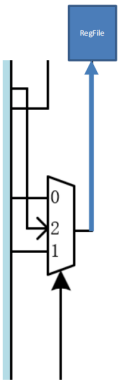
2024/5/8 Chapter 9 8

访存----功能介绍 ■ Memory Access(Mem)



- 存储器访问：存储器访问阶段涉及数据存储器（D_Mem）
- Load/Store指令对数据存储器进行读或写
- Mem_reg_WB：暂存存储器结果和控制信号，以待下一级使用

写回—功能介绍 ■ Write Back(WB)



- 写回：写回阶段涉及寄存器堆（RegisterFiles）；将ALU的运算结果、存储器输出结果、PC+4写回到寄存器堆。
- 写回阶段结束，一次完整的五级流水操作完成；此时下一次操作进行到存储器访问阶段（如果有）。由于在各级流水线之间插入了寄存器作为数据及控制信号的暂存，从而实现多条指令的重叠而不受影响。

2.4. 冒险与 stall

2.4.1. 实验目标及任务

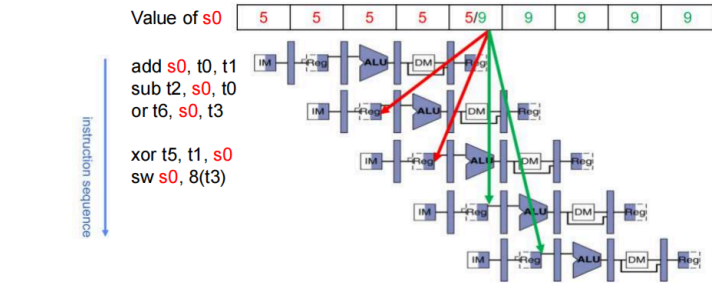
集成设计利用 stall 解决冒险的流水线 CPU，在 lab5-2 的基础上完成

· 设计冒险检测及 stall 消除冒险的流水线 CPU · 替换 lab5-2 的 CPU 为本实验集成的带 stall 处理的流水线 CPU

2.4.2. 实验原理

Data Hazard --Problem

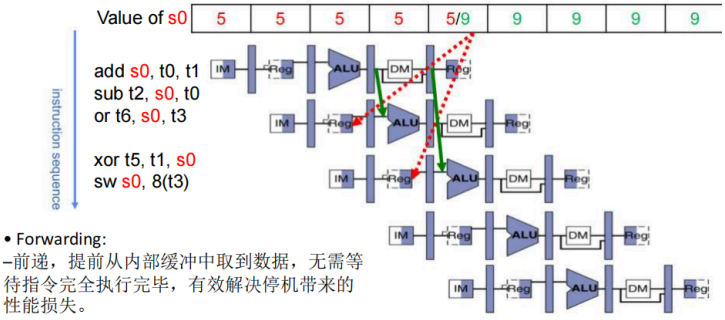
RAW数据冒险



- 指令SUB和OR的源操作数都依赖于ADD的计算结果，若不采取应对措施，则在S0结果写回之前，错误的值会被提前读走

Data Hazard---- Solution 3: Forwarding

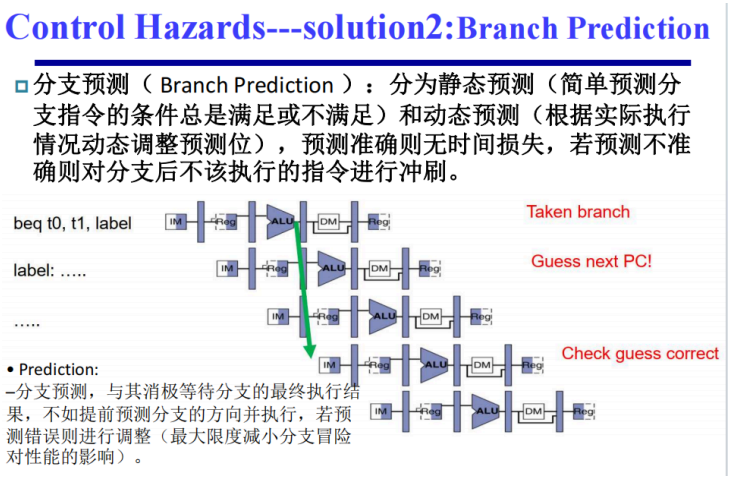
- 硬件解决方法：前递（forwarding），又称为旁路（bypassing）将数据通路生成的中间数据直接往前传递到ALU的输入端，参与下一条指令的运算



• Forwarding:
–前递，提前从内部缓冲中取到数据，无需等待指令完全执行完毕，有效解决停机带来的性能损失。

Control Hazards----problem

- 控制冒险：当PC值不按顺序执行时，流水线中指令的正常执行会被阻塞。
- 分支、跳转等引起的控制冒险（分支冒险）
- 异常、中断等引起的控制冒险



三、 实验实现方法、步骤与调试

3.1. 五级流水线的搭建与集成

(以下关于 ALU 的连线为 3 位，在实际仿真调试中均改为 4 位）

3.1.1. 取址

编写取指部分数据通路代码

```
module Pipeline_IF(
    input clk_IF,
    input rst_IF,
    input en_IF,
    input [31:0] PC_in_IF,
    input PCSrc,
    output reg [31:0] PC_out_IF
);

    wire [31:0] add_32_c;
    wire [31:0] MUX2T1_32_0_o;
    wire [31:0] REG32_Q;

    MUX2T1_32 MUX2T1_32_0(
        .I0(add_32_c),
        .I1(PC_in_IF),
        .s(PCSrc),
        .o(MUX2T1_32_0_o)
    );


    PC PC_U(
        .clk(clk_IF),
        .rst(rst_IF),
        .CE(en_IF),
        .D(MUX2T1_32_0_o),
        .Q(REG32_Q)
    );
```

```
    add_32 add_32_0(
        .a(32'd4),
        .b(REG32_Q),
        .c(add_32_c)
    );

    always @(*) begin
        PC_out_IF = REG32_Q;
    end

endmodule
```

编写 IF/ID 流水线寄存器代码

```
module IF_reg_ID(
    input clk_IFID,
    input rst_IFID,
    input en_IFID,
    input [31:0] PC_in_IFID,
    input [31:0] inst_in_IFID,
    output reg [31:0] PC_out_IFID,
    output reg [31:0] inst_out_IFID
);

    always @(posedge clk_IFID or posedge rst_IFID) begin
        if (rst_IFID == 1'b1) begin
            PC_out_IFID <= 32'b0;
            inst_out_IFID <= 32'b0;
        end
        else if (en_IFID == 1'b1) begin
            PC_out_IFID <= PC_in_IFID;
            inst_out_IFID <= inst_in_IFID;
        end
    end

endmodule
```

3.1.2. 译码

编写译码部分数据通路的代码

```
module Pipeline_ID(
    input clk_ID,
    input rst_ID,
    input RegWrite_in_ID,
    input [4:0] Rd_addr_ID,
    input [31:0] Wt_data_ID,
    input [31:0] Inst_in_ID,

    output reg [4:0] Rd_addr_out_ID,
    output reg [31:0] Rs1_out_ID,
    output reg [31:0] Rs2_out_ID,
    output reg [31:0] Imm_out_ID,
```

```
    output reg ALUSrc_B_ID,
    output reg [2:0] ALU_control_ID,
    output reg Branch_ID,
    output reg BranchN_ID,
    output reg MemRW_ID,
    output reg Jump_ID,
    output reg [1:0] MemtoReg_ID,
    output reg RegWrite_out_ID
);

    wire [31:0] SCPU_ctrl_ImmSel;
    wire [31:0] Regs_Rs1_data;
    wire [31:0] Regs_Rs2_data;
    wire [31:0] ImmGen_out;
    wire SCPU_ctrl_ALUSrc_B;
    wire [1:0] SCPU_ctrl_MemtoReg;
    wire SCPU_ctrl_Jump;
    wire SCPU_ctrl_Branch;
    wire SCPU_ctrl_BranchN;
    wire SCPU_ctrl_RegWrite;
    wire SCPU_ctrl_MemRW;
    wire [2:0] SCPU_ctrl_ALU_Control;

    // 实例化寄存器堆
    Register_new Regs_0(
        .clk(clk_ID),
        .rst(rst_ID),
        .Rs1_addr(Inst_in_ID[19:15]),
        .Rs2_addr(Inst_in_ID[24:20]),
        .Wt_addr(Rd_addr_ID),
        .Wt_data(Wt_data_ID),
        .RegWrite(RegWrite_in_ID),
        .Rs1_data(Regs_Rs1_data),
        .Rs2_data(Regs_Rs2_data)
    );

    // 实例化立即数生成器
    ImmGen_more ImmGen_0(
        .ImmSel(SCPU_ctrl_ImmSel),
        .inst_field(Inst_in_ID),
        .Imm_out(ImmGen_out)
    );

    // 实例化控制器
    SCPU_ctrl_more SCPU_ctrl_0(
        .OPcode(Inst_in_ID[6:2]),
        .Fun3(Inst_in_ID[14:12]),
        .Fun7(Inst_in_ID[30]),
        .ImmSel(SCPU_ctrl_ImmSel),
        .ALUSrc_B(SCPU_ctrl_ALUSrc_B),
        .MemtoReg(SCPU_ctrl_MemtoReg),
        .Jump(SCPU_ctrl_Jump),
```

```
        .Branch(SCPU_ctrl_Branch),
        .BranchN(SCPU_ctrl_BranchN),
        .RegWrite(SCPU_ctrl_RegWrite),
        .MemRW(SCPU_ctrl_MemRW),
        .ALU_Control(SCPU_ctrl_ALU_Control)
        //.CPU_MIO()
    );

    always @(*) begin
        Rs1_out_ID = Regs_Rs1_data;
        Rs2_out_ID = Regs_Rs2_data;
        Imm_out_ID = ImmGen_out;
        ALUSrc_B_ID = SCPU_ctrl_ALUSrc_B;
        MemtoReg_ID = SCPU_ctrl_MemtoReg;
        Jump_ID = SCPU_ctrl_Jump;
        Branch_ID = SCPU_ctrl_Branch;
        BranchN_ID = SCPU_ctrl_BranchN;
        RegWrite_out_ID = SCPU_ctrl_RegWrite;
        MemRW_ID = SCPU_ctrl_MemRW;
        ALU_control_ID = SCPU_ctrl_ALU_Control;
        Rd_addr_out_ID = Inst_in_ID[11:7];
    end

endmodule
```

编写 ID/EX 流水线寄存器的代码

```
module ID_reg_Ex(
    input clk_IDEX,
    input rst_IDEX,
    input en_IDEX,

    input [31:0] PC_in_IDEX,
    input [4:0] Rd_addr_IDEX,
    input [31:0] Rs1_in_IDEX,
    input [31:0] Rs2_in_IDEX,
    input [31:0] Imm_in_IDEX,
    input ALUSrc_B_in_IDEX,
    input [2:0] ALU_control_in_IDEX,
    input Branch_in_IDEX,
    input BranchN_in_IDEX,
    input MemRW_in_IDEX,
    input Jump_in_IDEX,
    input [1:0] MemtoReg_in_IDEX,
    input RegWrite_in_IDEX,

    output reg [31:0] PC_out_IDEX,
    output reg [4:0] Rd_addr_out_IDEX,
    output reg [31:0] Rs1_out_IDEX,
    output reg [31:0] Rs2_out_IDEX,
    output reg [31:0] Imm_out_IDEX,
    output reg ALUSrc_B_out_IDEX,
    output reg [2:0] ALU_control_out_IDEX,
```

```
    output reg Branch_out_IDEX,
    output reg BranchN_out_IDEX,
    output reg MemRW_out_IDEX,
    output reg Jump_out_IDEX,
    output reg [1:0] MemtoReg_out_IDEX,
    output reg RegWrite_out_IDEX
);

    always @(posedge clk_IDEX or posedge rst_IDEX) begin
        if (rst_IDEX == 1'b1) begin
            PC_out_IDEX <= 32'b0;
            Rd_addr_out_IDEX <= 5'b0;
            Rs1_out_IDEX <= 32'b0;
            Rs2_out_IDEX <= 32'b0;
            Imm_out_IDEX <= 32'b0;
            ALUSrc_B_out_IDEX <= 1'b0;
            ALU_control_out_IDEX <= 3'b0;
            Branch_out_IDEX <= 1'b0;
            BranchN_out_IDEX <= 1'b0;
            MemRW_out_IDEX <= 1'b0;
            Jump_out_IDEX <= 1'b0;
            MemtoReg_out_IDEX <= 2'b0;
            RegWrite_out_IDEX <= 1'b0;
        end
        else if (en_IDEX == 1'b1) begin
            PC_out_IDEX <= PC_in_IDEX;
            Rd_addr_out_IDEX <= Rd_addr_IDEX;
            Rs1_out_IDEX <= Rs1_in_IDEX;
            Rs2_out_IDEX <= Rs2_in_IDEX;
            Imm_out_IDEX <= Imm_in_IDEX;
            ALUSrc_B_out_IDEX <= ALUSrc_B_in_IDEX;
            ALU_control_out_IDEX <= ALU_control_in_IDEX;
            Branch_out_IDEX <= Branch_in_IDEX;
            BranchN_out_IDEX <= BranchN_in_IDEX;
            MemRW_out_IDEX <= MemRW_in_IDEX;
            Jump_out_IDEX <= Jump_in_IDEX;
            MemtoReg_out_IDEX <= MemtoReg_in_IDEX;
            RegWrite_out_IDEX <= RegWrite_in_IDEX;
        end
    end

endmodule
```

3.1.3. 执行

编写执行部分数据通路的代码

```
module Pipeline_Ex(
    input [31:0] PC_in_EX,
    input [31:0] Rs1_in_EX,
    input [31:0] Rs2_in_EX,
    input [31:0] Imm_in_EX,
    input ALUSrc_B_in_EX,
```

```
    input [2:0] ALU_control_in_EX,

    output reg [31:0] PC_out_EX,
    output reg [31:0] PC4_out_EX,
    output reg zero_out_EX,
    output reg [31:0] ALU_out_EX,
    output reg [31:0] Rs2_out_EX
);

    wire [31:0] MUX2T1_32_0_o;
    wire [31:0] add_32_0_c;
    wire [31:0] add_32_1_c;
    wire [31:0] ALU_res;
    wire ALU_zero;

    // 计算 PC + 4
    add_32 add_32_1(
        .a(32'd4),
        .b(PC_in_EX),
        .c(add_32_1_c)
    );

    // 计算 PC + Imm (用于分支/跳转目标地址)
    add_32 add_32_0(
        .a(PC_in_EX),
        .b(Imm_in_EX),
        .c(add_32_0_c)
    );

    // ALU 输入 B 选择 (寄存器数据 或 立即数)
    MUX2T1_32 MUX2T1_32_0(
        .I0(Rs2_in_EX),
        .I1(Imm_in_EX),
        .s(ALUSrc_B_in_EX),
        .o(MUX2T1_32_0_o)
    );

    // ALU 运算
    ALU ALU(
        .A(Rs1_in_EX),
        .ALU_operation(ALU_control_in_EX),
        .B(MUX2T1_32_0_o),
        .res(ALU_res),
        .zero(ALU_zero)
    );

    always @(*) begin
        PC4_out_EX = add_32_1_c;
        PC_out_EX = add_32_0_c;
        ALU_out_EX = ALU_res;
        zero_out_EX = ALU_zero;
        Rs2_out_EX = Rs2_in_EX;
```

```
    end

endmodule
```

编写 EX/Mem 流水线寄存器代码

```
module Ex_reg_Mem(
    input clk_EXMem,
    input rst_EXMem,
    input en_EXMem,

    input [31:0] PC_in_EXMem,
    input [31:0] PC4_in_EXMem,
    input [4:0] Rd_addr_EXMem,
    input zero_in_EXMem,
    input [31:0] ALU_in_EXMem,
    input [31:0] Rs2_in_EXMem,
    input Branch_in_EXMem,
    input BranchN_in_EXMem,
    input MemRW_in_EXMem,
    input Jump_in_EXMem,
    input [1:0] MemtoReg_in_EXMem,
    input RegWrite_in_EXMem,

    output reg [31:0] PC_out_EXMem,
    output reg [31:0] PC4_out_EXMem,
    output reg [4:0] Rd_addr_out_EXMem,
    output reg zero_out_EXMem,
    output reg [31:0] ALU_out_EXMem,
    output reg [31:0] Rs2_out_EXMem,
    output reg Branch_out_EXMem,
    output reg BranchN_out_EXMem,
    output reg MemRW_out_EXMem,
    output reg Jump_out_EXMem,
    output reg [1:0] MemtoReg_out_EXMem,
    output reg RegWrite_out_EXMem
);

    always @(posedge clk_EXMem or posedge rst_EXMem) begin
        if (rst_EXMem == 1'b1) begin
            PC_out_EXMem <= 32'b0;
            PC4_out_EXMem <= 32'b0;
            Rd_addr_out_EXMem <= 5'b0;
            zero_out_EXMem <= 1'b0;
            ALU_out_EXMem <= 32'b0;
            Rs2_out_EXMem <= 32'b0;
            Branch_out_EXMem <= 1'b0;
            BranchN_out_EXMem <= 1'b0;
            MemRW_out_EXMem <= 1'b0;
            Jump_out_EXMem <= 1'b0;
            MemtoReg_out_EXMem <= 2'b0;
            RegWrite_out_EXMem <= 1'b0;
        end
```

```
        else if (en_EXMem == 1'b1) begin
            PC_out_EXMem <= PC_in_EXMem;
            PC4_out_EXMem <= PC4_in_EXMem;
            Rd_addr_out_EXMem <= Rd_addr_EXMem;
            zero_out_EXMem <= zero_in_EXMem;
            ALU_out_EXMem <= ALU_in_EXMem;
            Rs2_out_EXMem <= Rs2_in_EXMem;
            Branch_out_EXMem <= Branch_in_EXMem;
            BranchN_out_EXMem <= BranchN_in_EXMem;
            MemRW_out_EXMem <= MemRW_in_EXMem;
            Jump_out_EXMem <= Jump_in_EXMem;
            MemtoReg_out_EXMem <= MemtoReg_in_EXMem;
            RegWrite_out_EXMem <= RegWrite_in_EXMem;
        end
    end

endmodule
```

3.1.4. 访存

编写访存部分数据通路代码

```
module Pipeline_Mem(
    input zero_in_Mem,        // zero 信号
    input Branch_in_Mem,      // beq 信号
    input BranchN_in_Mem,     // bne 信号
    input Jump_in_Mem,        // jal 信号
    output PCSrc              // PC 选择控制输出
);

    assign PCSrc = Jump_in_Mem | (Branch_in_Mem & zero_in_Mem) | (BranchN_in_Mem &
~zero_in_Mem);

endmodule
```

编写 Mem/WB 流水线寄存器代码

```
module Mem_reg_WB(
    input clk_MemWB,
    input rst_MemWB,
    input en_MemWB,

    input [31:0] PC4_in_MemWB,
    input [4:0] Rd_addr_MemWB,
    input [31:0] ALU_in_MemWB,
    input [31:0] DMem_data_MemWB,
    input [1:0] MemtoReg_in_MemWB,
    input RegWrite_in_MemWB,

    output reg [31:0] PC4_out_MemWB,
    output reg [4:0] Rd_addr_out_MemWB,
    output reg [31:0] ALU_out_MemWB,
    output reg [31:0] DMem_data_out_MemWB,
```

```
    output reg [1:0] MemtoReg_out_MemWB,
    output reg RegWrite_out_MemWB
);

    always @(posedge clk_MemWB or posedge rst_MemWB) begin
        if (rst_MemWB == 1'b1) begin
            PC4_out_MemWB <= 32'b0;
            Rd_addr_out_MemWB <= 5'b0;
            ALU_out_MemWB <= 32'b0;
            DMem_data_out_MemWB <= 32'b0;
            MemtoReg_out_MemWB <= 2'b0;
            RegWrite_out_MemWB <= 1'b0;
        end
        else if (en_MemWB == 1'b1) begin
            PC4_out_MemWB <= PC4_in_MemWB;
            Rd_addr_out_MemWB <= Rd_addr_MemWB;
            ALU_out_MemWB <= ALU_in_MemWB;
            DMem_data_out_MemWB <= DMem_data_MemWB;
            MemtoReg_out_MemWB <= MemtoReg_in_MemWB;
            RegWrite_out_MemWB <= RegWrite_in_MemWB;
        end
    end

endmodule
```

3.1.5. 写回

编写写回部分数据通路的代码

```
module Pipeline_WB(
    input [31:0] PC4_in_WB,
    input [31:0] ALU_in_WB,
    input [31:0] DMem_data_WB,
    input [1:0] MemtoReg_in_WB,
    output [31:0] Data_out_WB
);

    MUX4T1_32 MUX4T1_32_0(
        .s(MemtoReg_in_WB),
        .I0(ALU_in_WB),
        .I1(DMem_data_WB),
        .I2(PC4_in_WB),
        .I3(PC4_in_WB),
        .o(Data_out_WB)
    );

endmodule
```

3.1.6. 流水线 CPU

编写流水线 CPU 代码，并利用前面写好的模块进行集成

```
module Pipeline_CPU(
    input clk,                      // 时钟
```

```
    input rst,                      // 复位
    input [31:0] Data_in,           // 存储器数据输入
    input [31:0] inst_IF,           // 取指阶段指令
    output [31:0] PC_out_IF,        // 取指阶段 PC输出
    output [31:0] PC_out_ID,        // 译码阶段 PC输出
    output [31:0] inst_ID,          // 译码阶段指令
    output [31:0] PC_out_EX,        // 执行阶段 PC输出
    output [31:0] MemRW_EX,         // 执行阶段存储器读写
    output [31:0] MemRW_Mem,        // 访存阶段存储器读写
    output [31:0] Addr_out,         // 地址输出
    output [31:0] Data_out,         // CPU数据输出
    output [31:0] Data_out_WB       // 写回数据输出
);

    wire Pipeline_Mem_PCSrc;
    wire [31:0] Pipeline_IF_PC_out_IF;
    wire [31:0] IF_reg_ID_PC_out_IFID;
    wire [31:0] IF_reg_ID_inst_out_IFID;
    wire [31:0] Pipeline_WB_Data_out_WB;
    wire [31:0] Pipeline_ID_Rd_addr_out_ID;
    wire [31:0] Pipeline_ID_Rs1_out_ID;
    wire [31:0] Pipeline_ID_Rs2_out_ID;
    wire [31:0] Pipeline_ID_Imm_out_ID;
    wire Pipeline_ID_ALUSrc_B_ID;
    wire [2:0] Pipeline_ID_ALU_control_ID;
    wire Pipeline_ID_Branch_ID;
    wire Pipeline_ID_BranchN_ID;
    wire Pipeline_ID_MemRW_ID;
    wire Pipeline_ID_Jump_ID;
    wire [1:0] Pipeline_ID_MemtoReg_ID;
    wire Pipeline_ID_RegWrite_out_ID;
    wire [31:0] ID_reg_Ex_PC_out_IDEX;
    wire [4:0] ID_reg_Ex_Rd_addr_out_IDEX;
    wire [31:0] ID_reg_Ex_Rs1_out_IDEX;
    wire [31:0] ID_reg_Ex_Rs2_out_IDEX;
    wire [31:0] ID_reg_Ex_Imm_out_IDEX;
    wire ID_reg_Ex_ALUSrc_B_out_IDEX;
    wire [2:0] ID_reg_Ex_ALU_control_out_IDEX;
    wire ID_reg_Ex_Branch_out_IDEX;
    wire ID_reg_Ex_BranchN_out_IDEX;
    wire ID_reg_Ex_MemRW_out_IDEX;
    wire ID_reg_Ex_Jump_out_IDEX;
    wire [1:0] ID_reg_Ex_MemtoReg_out_IDEX;
    wire ID_reg_Ex_RegWrite_out_IDEX;
    wire [31:0] Pipeline_Ex_PC_out_EX;
    wire [31:0] Pipeline_Ex_PC4_out_EX;
    wire Pipeline_Ex_zero_out_EX;
    wire [31:0] Pipeline_Ex_ALU_out_EX;
    wire [31:0] Pipeline_Ex_Rs2_out_EX;
    wire [31:0] Ex_reg_Mem_PC_out_EXMem;
    wire [31:0] Ex_reg_Mem_PC4_out_EXMem;
    wire [4:0] Ex_reg_Mem_Rd_addr_out_EXMem;
```

```
wire Ex_reg_Mem_zero_out_EXMem;
wire [31:0] Ex_reg_Mem_ALU_out_EXMem;
wire [31:0] Ex_reg_Mem_Rs2_out_EXMem;
wire Ex_reg_Mem_Branch_out_EXMem;
wire Ex_reg_Mem_BranchN_out_EXMem;
wire Ex_reg_Mem_MemRW_out_EXMem;
wire Ex_reg_Mem_Jump_out_EXMem;
wire [1:0] Ex_reg_Mem_MemtoReg_out_EXMem;
wire Ex_reg_Mem_RegWrite_out_EXMem;
wire [31:0] Mem_reg_WB_PC4_out_MemWB;
wire [4:0] Mem_reg_WB_Rd_addr_out_MemWB;
wire [31:0] Mem_reg_WB_ALU_out_MemWB;
wire [31:0] Mem_reg_WB_DMem_data_out_MemWB;
wire [1:0] Mem_reg_WB_MemtoReg_out_MemWB;
wire Mem_reg_WB_RegWrite_out_MemWB;

Pipeline_IF Pipeline_CPU_IF(
    .clk_IF(clk),
    .rst_IF(rst),
    .en_IF(1'b1),
    .PC_in_IF(Ex_reg_Mem_PC_out_EXMem),
    .PCSrc (Pipeline_Mem_PCSrc),
    .PC_out_IF (Pipeline_IF_PC_out_IF)
);

IF_reg_ID Pipeline_IF_reg_ID(
    .clk_IFID(clk),
    .rst_IFID(rst),
    .en_IFID(1'b1),
    .PC_in_IFID (Pipeline_IF_PC_out_IF),
    .inst_in_IFID (inst_IF),
    .PC_out_IFID (IF_reg_ID_PC_out_IFID),
    .inst_out_IFID (IF_reg_ID_inst_out_IFID)
);

Pipeline_ID Pipeline_CPU_ID(
    .clk_ID(clk),
    .rst_ID(rst),
    .RegWrite_in_ID (Mem_reg_WB_RegWrite_out_MemWB),
    .Rd_addr_ID (Mem_reg_WB_Rd_addr_out_MemWB),
    .Wt_data_ID (Pipeline_WB_Data_out_WB),
    .Inst_in_ID (IF_reg_ID_inst_out_IFID),
    .Rd_addr_out_ID (Pipeline_ID_Rd_addr_out_ID),
    .Rs1_out_ID (Pipeline_ID_Rs1_out_ID),
    .Rs2_out_ID (Pipeline_ID_Rs2_out_ID),
    .Imm_out_ID (Pipeline_ID_Imm_out_ID),
    .ALUSrc_B_ID (Pipeline_ID_ALUSrc_B_ID),
    .ALU_control_ID (Pipeline_ID_ALU_control_ID),
    .Branch_ID (Pipeline_ID_Branch_ID),
    .BranchN_ID (Pipeline_ID_BranchN_ID),
    .MemRW_ID (Pipeline_ID_MemRW_ID),
    .Jump_ID (Pipeline_ID_Jump_ID),
    .MemtoReg_ID (Pipeline_ID_MemtoReg_ID),
```

```
    .RegWrite_out_ID (Pipeline_ID_RegWrite_out_ID)
);

ID_reg_Ex Pipeline_ID_reg_Ex(
    .clk_IDEX(clk),
    .rst_IDEX(rst),
    .en_IDEX(1'b1),
    .PC_in_IDEX(IF_reg_ID_PC_out_IFID),
    .Rd_addr_IDEX (Pipeline_ID_Rd_addr_out_ID),
    .Rs1_in_IDEX (Pipeline_ID_Rs1_out_ID),
    .Rs2_in_IDEX (Pipeline_ID_Rs2_out_ID),
    .Imm_in_IDEX (Pipeline_ID_Imm_out_ID),
    .ALUSrc_B_in_IDEX (Pipeline_ID_ALUSrc_B_ID),
    .ALU_control_in_IDEX(Pipeline_ID_ALU_control_ID),
    .Branch_in_IDEX (Pipeline_ID_Branch_ID),
    .BranchN_in_IDEX (Pipeline_ID_BranchN_ID),
    .MemRW_in_IDEX (Pipeline_ID_MemRW_ID),
    .Jump_in_IDEX (Pipeline_ID_Jump_ID),
    .MemtoReg_in_IDEX (Pipeline_ID_MemtoReg_ID),
    .RegWrite_in_IDEX (Pipeline_ID_RegWrite_out_ID),
    .PC_out_IDEX(ID_reg_Ex_PC_out_IDEX),
    .Rd_addr_out_IDEX(ID_reg_Ex_Rd_addr_out_IDEX),
    .Rs1_out_IDEX(ID_reg_Ex_Rs1_out_IDEX),
    .Rs2_out_IDEX(ID_reg_Ex_Rs2_out_IDEX),
    .Imm_out_IDEX(ID_reg_Ex_Imm_out_IDEX),
    .ALUSrc_B_out_IDEX(ID_reg_Ex_ALUSrc_B_out_IDEX),
    .ALU_control_out_IDEX(ID_reg_Ex_ALU_control_out_IDEX),
    .Branch_out_IDEX(ID_reg_Ex_Branch_out_IDEX),
    .BranchN_out_IDEX(ID_reg_Ex_BranchN_out_IDEX),
    .MemRW_out_IDEX(ID_reg_Ex_MemRW_out_IDEX),
    .Jump_out_IDEX(ID_reg_Ex_Jump_out_IDEX),
    .MemtoReg_out_IDEX(ID_reg_Ex_MemtoReg_out_IDEX),
    .RegWrite_out_IDEX(ID_reg_Ex_RegWrite_out_IDEX)
);

Pipeline_Ex Pipeline_CPU_Ex(
    .PC_in_EX(ID_reg_Ex_PC_out_IDEX),
    .Rs1_in_EX(ID_reg_Ex_Rs1_out_IDEX),
    .Rs2_in_EX(ID_reg_Ex_Rs2_out_IDEX),
    .Imm_in_EX(ID_reg_Ex_Imm_out_IDEX),
    .ALUSrc_B_in_EX(ID_reg_Ex_ALUSrc_B_out_IDEX),
    .ALU_control_in_EX(ID_reg_Ex_ALU_control_out_IDEX),
    .PC_out_EX(Pipeline_Ex_PC_out_EX),
    .PC4_out_EX (Pipeline_Ex_PC4_out_EX),
    .zero_out_EX(Pipeline_Ex_zero_out_EX),
    .ALU_out_EX (Pipeline_Ex_ALU_out_EX),
    .Rs2_out_EX (Pipeline_Ex_Rs2_out_EX)
);

Ex_reg_Mem Pipeline_Ex_reg_Mem(
    .clk_EXMem(clk),
    .rst_EXMem(rst),
    .en_EXMem(1'b1),
```

```
    .PC_in_EXMem (Pipeline_Ex_PC_out_EX),
    .PC4_in_EXMem (Pipeline_Ex_PC4_out_EX),
    .Rd_addr_EXMem (ID_reg_Ex_Rd_addr_out_IDEX),
    .zero_in_EXMem (Pipeline_Ex_zero_out_EX),
    .ALU_in_EXMem (Pipeline_Ex_ALU_out_EX),
    .Rs2_in_EXMem (Pipeline_Ex_Rs2_out_EX),
    .Branch_in_EXMem(ID_reg_Ex_Branch_out_IDEX),
    .BranchN_in_EXMem (ID_reg_Ex_BranchN_out_IDEX),
    .MemRW_in_EXMem (ID_reg_Ex_MemRW_out_IDEX),
    .Jump_in_EXMem (ID_reg_Ex_Jump_out_IDEX),
    .MemtoReg_in_EXMem(ID_reg_Ex_MemtoReg_out_IDEX),
    .RegWrite_in_EXMem(ID_reg_Ex_RegWrite_out_IDEX),
    .PC_out_EXMem (Ex_reg_Mem_PC_out_EXMem),
    .PC4_out_EXMem (Ex_reg_Mem_PC4_out_EXMem),
    .Rd_addr_out_EXMem (Ex_reg_Mem_Rd_addr_out_EXMem),
    .zero_out_EXMem (Ex_reg_Mem_zero_out_EXMem),
    .ALU_out_EXMem (Ex_reg_Mem_ALU_out_EXMem),
    .Rs2_out_EXMem (Ex_reg_Mem_Rs2_out_EXMem),
    .Branch_out_EXMem (Ex_reg_Mem_Branch_out_EXMem),
    .BranchN_out_EXMem (Ex_reg_Mem_BranchN_out_EXMem),
    .MemRW_out_EXMem (Ex_reg_Mem_MemRW_out_EXMem),
    .Jump_out_EXMem (Ex_reg_Mem_Jump_out_EXMem),
    .MemtoReg_out_EXMem (Ex_reg_Mem_MemtoReg_out_EXMem),
    .RegWrite_out_EXMem (Ex_reg_Mem_RegWrite_out_EXMem)
);

Pipeline_Mem Pipeline_CPU_Mem(
    .zero_in_Mem (Ex_reg_Mem_zero_out_EXMem),
    .Branch_in_Mem (Ex_reg_Mem_Branch_out_EXMem),
    .BranchN_in_Mem (Ex_reg_Mem_BranchN_out_EXMem),
    .Jump_in_Mem (Ex_reg_Mem_Jump_out_EXMem),
    .PCSrc (Pipeline_Mem_PCSrc)
);

Mem_reg_WB Pipeline_Mem_reg_WB(
    .clk_MemWB(clk),
    .rst_MemWB(rst),
    .en_MemWB(1'b1),
    .PC4_in_MemWB(Ex_reg_Mem_PC4_out_EXMem),
    .Rd_addr_MemWB (Ex_reg_Mem_Rd_addr_out_EXMem),
    .ALU_in_MemWB(Ex_reg_Mem_ALU_out_EXMem),
    .DMem_data_MemWB (Data_in),
    .MemtoReg_in_MemWB(Ex_reg_Mem_MemtoReg_out_EXMem),
    .RegWrite_in_MemWB(Ex_reg_Mem_RegWrite_out_EXMem),
    .PC4_out_MemWB (Mem_reg_WB_PC4_out_MemWB),
    .Rd_addr_out_MemWB(Mem_reg_WB_Rd_addr_out_MemWB),
    .ALU_out_MemWB(Mem_reg_WB_ALU_out_MemWB),
    .DMem_data_out_MemWB (Mem_reg_WB_DMem_data_out_MemWB),
    .MemtoReg_out_MemWB (Mem_reg_WB_MemtoReg_out_MemWB),
    .RegWrite_out_MemWB (Mem_reg_WB_RegWrite_out_MemWB)
);

Pipeline_WB Pipeline_CPU_WB(
```

```
        .PC4_in_WB (Mem_reg_WB_PC4_out_MemWB),
        .ALU_in_WB (Mem_reg_WB_ALU_out_MemWB),
        .DMem_data_WB(Mem_reg_WB_DMem_data_out_MemWB),
        .MemtoReg_in_WB (Mem_reg_WB_MemtoReg_out_MemWB),
        .Data_out_WB (Pipeline_WB_Data_out_WB)
    );

    assign PC_out_EX = Pipeline_Ex_PC_out_EX;
    assign PC_out_ID = IF_reg_ID_PC_out_IFID;
    assign inst_ID = IF_reg_ID_inst_out_IFID;
    assign PC_out_IF = Pipeline_IF_PC_out_IF;
    assign Addr_out = Ex_reg_Mem_ALU_out_EXMem;
    assign Data_out = Ex_reg_Mem_Rs2_out_EXMem;
    assign Data_out_WB = Pipeline_WB_Data_out_WB;
    assign MemRW_Mem = Ex_reg_Mem_MemRW_out_EXMem;
    assign MemRW_EX = ID_reg_Ex_MemRW_out_IDEX;

endmodule
```

3.1.7. 改进测试平台

改进测试平台代码，并分别用提供的 noHazard.mem 和 Hazard_NOP.mem 文件作为 ROM 指令进行仿真，仿真结果见 四、实验结果和分析

```
module soc_test_wrapper (
    input clk,
    input rst
);

    wire [31:0] RAM_B_douta;
    wire [31:0] CPU_inst_in;
    wire [31:0] CPU_Addr_out;
    wire CPU_MemRW;
    wire [31:0] CPU_Data_out;
    wire [31:0] CPU_PC_out;
    wire [9:0] Addr_out_slice;
    wire [9:0] PC_out_slice;
    wire [31:0] CPU_PC_out_ID;
    wire [31:0] CPU_PC_out_Ex;
    wire [31:0] CPU_inst_ID;
    wire CPU_MemRW_Ex;

    RAM RAM_0(
        .clka(~clk),
        .wea(CPU_MemRW),
        .addra(Addr_out_slice),
        .dina(CPU_Data_out),
        .douta(RAM_B_douta)
    );

    ROM_1 ROM_0(
        .a(PC_out_slice),
        .spo(CPU_inst_in)
```

```
    );

    Pipeline_CPU Pipeline_CPU_wrapper_0(
        .clk(clk),
        .rst(rst),
        .Data_in(RAM_B_douta),
        .inst_IF(CPU_inst_in),
        .PC_out_IF(CPU_PC_out),
        .PC_out_ID(CPU_PC_out_ID),
        .inst_ID(CPU_inst_ID),
        .PC_out_EX(CPU_PC_out_Ex),
        .MemRW_EX(CPU_MemRW_Ex),
        .MemRW_Mem(CPU_MemRW),
        .Addr_out(CPU_Addr_out),
        .Data_out(CPU_Data_out)
    );

    assign Addr_out_slice = CPU_Addr_out[11:2];
    assign PC_out_slice = CPU_PC_out[11:2];

endmodule
```

3.2. stall 机制处理流水线冒险

编写停顿代码

```
module stall(
    input rst_stall,

    input RegWrite_out_IDEX,
    input [4:0] Rd_addr_out_IDEX,
    input RegWrite_out_EXMem,
    input [4:0] Rd_addr_out_EXMem,
    input RegWrite_out_MemWB,
    input [4:0] Rd_addr_out_MemWB,
    input [4:0] Rs1_addr_ID,
    input [4:0] Rs2_addr_ID,
    input Rs1_used,
    input Rs2_used,

    input PCSrc,

    output NOP_EXMem,
    output en_IF,
    output en_IFID,
    output NOP_IFID,
    output NOP_IDEX
);
    reg Data_stall;

    always @(*) begin
        Data_stall = 0;
        if (RegWrite_out_IDEX && Rs1_used && Rs1_addr_ID != 0 && Rd_addr_out_IDEX ==
```

```
Rs1_addr_ID) Data_stall = 1;
        else if (RegWrite_out_IDEX && Rs2_used && Rs2_addr_ID != 0 &&
Rd_addr_out_IDEX == Rs2_addr_ID) Data_stall = 1;
        else if (RegWrite_out_EXMem && Rs1_used && Rs1_addr_ID != 0 &&
Rd_addr_out_EXMem == Rs1_addr_ID) Data_stall = 1;
        else if (RegWrite_out_EXMem && Rs2_used && Rs2_addr_ID != 0 &&
Rd_addr_out_EXMem == Rs2_addr_ID) Data_stall = 1;
        else if (RegWrite_out_MemWB && Rs1_used && Rs1_addr_ID != 0 &&
Rd_addr_out_MemWB == Rs1_addr_ID) Data_stall = 1;
        else if (RegWrite_out_MemWB && Rs2_used && Rs2_addr_ID != 0 &&
Rd_addr_out_MemWB == Rs2_addr_ID) Data_stall = 1;
    end


    assign en_IF    = ~Data_stall;  // 数据冒险时停顿 PC
    assign en_IFID  = ~Data_stall;  // 数据冒险时停顿 IF/ID

    assign NOP_IDEX = Data_stall || PCSrc;

    assign NOP_IFID = PCSrc;

    assign NOP_EXMem = PCSrc;

endmodule
```

对其他模块代码进行修改

1.

```
module Ex_reg_Mem(
    input clk_EXMem,
    input rst_EXMem,
    input en_EXMem,
    input [31:0] PC_in_EXMem,
    input [31:0] PC4_in_EXMem,
    input [4:0] Rd_addr_EXMem,
    input zero_in_EXMem,
    input [31:0] ALU_in_EXMem,
    input [31:0] Rs2_in_EXMem,
    input Branch_in_EXMem,
    input BranchN_in_EXMem,
    input MemRW_in_EXMem,
    input Jump_in_EXMem,
    input [1:0] MemtoReg_in_EXMem,
    input RegWrite_in_EXMem,

    input NOP_EXMem,

    output reg [31:0] PC_out_EXMem,
    output reg [31:0] PC4_out_EXMem,
    output reg [4:0] Rd_addr_out_EXMem,
    output reg zero_out_EXMem,
    output reg [31:0] ALU_out_EXMem,
    output reg [31:0] Rs2_out_EXMem,
```

```
    output reg Branch_out_EXMem,
    output reg BranchN_out_EXMem,
    output reg MemRW_out_EXMem,
    output reg Jump_out_EXMem,
    output reg [1:0] MemtoReg_out_EXMem,
    output reg RegWrite_out_EXMem
);

    always @(posedge clk_EXMem or posedge rst_EXMem) begin
        if (rst_EXMem == 1'b1) begin
            PC_out_EXMem <= 32'b0;
            PC4_out_EXMem <= 32'b0;
            Rd_addr_out_EXMem <= 5'b0;
            zero_out_EXMem <= 1'b0;
            ALU_out_EXMem <= 32'b0;
            Rs2_out_EXMem <= 32'b0;
            Branch_out_EXMem <= 1'b0;
            BranchN_out_EXMem <= 1'b0;
            MemRW_out_EXMem <= 1'b0;
            Jump_out_EXMem <= 1'b0;
            MemtoReg_out_EXMem <= 2'b0;
            RegWrite_out_EXMem <= 1'b0;
        end
        else if (en_EXMem == 1'b1) begin
            if (NOP_EXMem == 1'b1) begin
                RegWrite_out_EXMem <= 1'b0; // 禁止写寄存器
                MemRW_out_EXMem <= 1'b0;    // 禁止写内存
                Branch_out_EXMem <= 1'b0;   // 禁止分支
                BranchN_out_EXMem <= 1'b0;
                Jump_out_EXMem <= 1'b0;     // 禁止跳转

                PC_out_EXMem <= 32'b0;

                PC4_out_EXMem <= 32'b0;
                Rd_addr_out_EXMem <= 5'b0;
                zero_out_EXMem <= 1'b0;
                ALU_out_EXMem <= 32'b0;
                Rs2_out_EXMem <= 32'b0;
                MemtoReg_out_EXMem <= 2'b0;
            end
            else begin
                PC_out_EXMem <= PC_in_EXMem;
                PC4_out_EXMem <= PC4_in_EXMem;
                Rd_addr_out_EXMem <= Rd_addr_EXMem;
                zero_out_EXMem <= zero_in_EXMem;
                ALU_out_EXMem <= ALU_in_EXMem;
                Rs2_out_EXMem <= Rs2_in_EXMem;
                Branch_out_EXMem <= Branch_in_EXMem;
                BranchN_out_EXMem <= BranchN_in_EXMem;
                MemRW_out_EXMem <= MemRW_in_EXMem;
                Jump_out_EXMem <= Jump_in_EXMem;
                MemtoReg_out_EXMem <= MemtoReg_in_EXMem;
```

```
                RegWrite_out_EXMem <= RegWrite_in_EXMem;
            end
        end
    end
endmodule
```

2.

```
module Register(
    input clk,
    input rst,
    input [4:0] Rs1_addr,
    input [4:0] Rs2_addr,
    input [4:0] Wt_addr,
    input [31:0] Wt_data,
    input RegWrite,
    output [31:0] Rs1_data,
    output [31:0] Rs2_data
);
    reg [31:0] regs [1:31];
    integer i;

    // 写逻辑（保持不变）
    always @(posedge clk or posedge rst) begin
        if (rst == 1) begin
            for (i = 1; i <= 31; i = i + 1) regs[i] <= 32'b0;
        end
        else if (RegWrite == 1 && Wt_addr != 0) begin
            regs[Wt_addr] <= Wt_data;
        end
    end

    // 读逻辑增加"内部前递"判断
    // 如果当前正在写的地址 == 正在读的地址，直接输出 Wt_data
    assign Rs1_data = (Rs1_addr == 0) ? 32'b0 :
                      (RegWrite && (Rs1_addr == Wt_addr)) ? Wt_data :
                      regs[Rs1_addr];

    assign Rs2_data = (Rs2_addr == 0) ? 32'b0 :
                      (RegWrite && (Rs2_addr == Wt_addr)) ? Wt_data :
                      regs[Rs2_addr];

endmodule
```

3.

```
// module Pipeline_CPU... (前略)

    // Stall 模块控制信号
    wire stall_en_IF;
    wire stall_en_IFID;
    wire stall_NOP_IDEX;
    wire stall_NOP_IFID;
    wire stall_NOP_EXMem;
```

```
    // ... (中间略)

    // 6. EX/MEM 流水线寄存器
    Ex_reg_Mem Pipeline_Ex_reg_Mem(
        .clk_EXMem(clk),
        .rst_EXMem(rst),
        .en_EXMem(1'b1),

        .NOP_EXMem(stall_NOP_EXMem),

        .PC_in_EXMem (Pipeline_Ex_PC_out_EX),
        // ... (其余保持不变) ...
    );

    // ... (中间略)

    // 10. 冒险与停顿处理模块 (Stall)
    stall Pipeline_stall(
        .rst_stall(rst),
        // ... (输入信号保持不变) ...

        .en_IF (stall_en_IF),
        .en_IFID(stall_en_IFID),
        .NOP_IDEX(stall_NOP_IDEX),
        .NOP_IFID(stall_NOP_IFID),
        .NOP_EXMem(stall_NOP_EXMem)
    );

// ... (略)
```

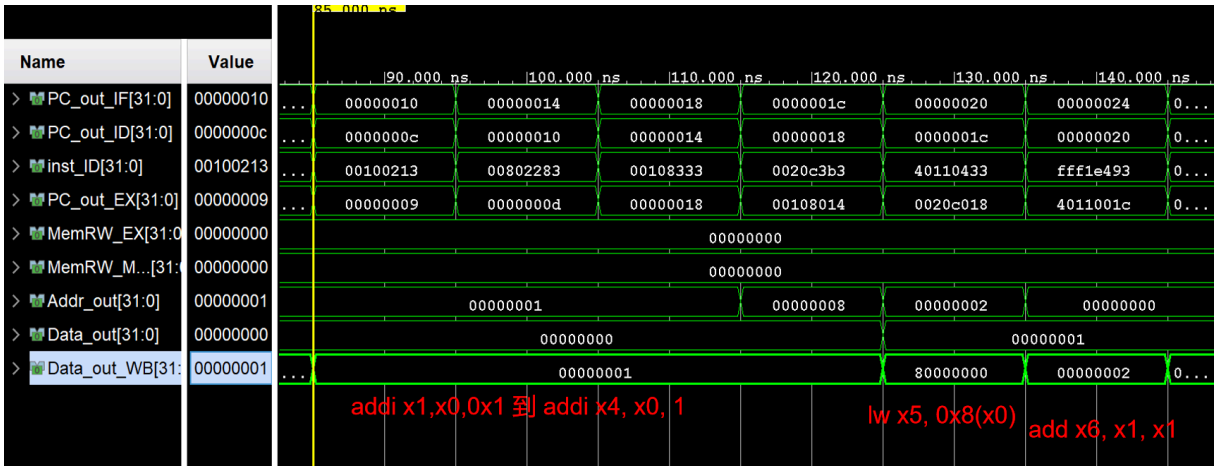
实验结果见实验结果与分析

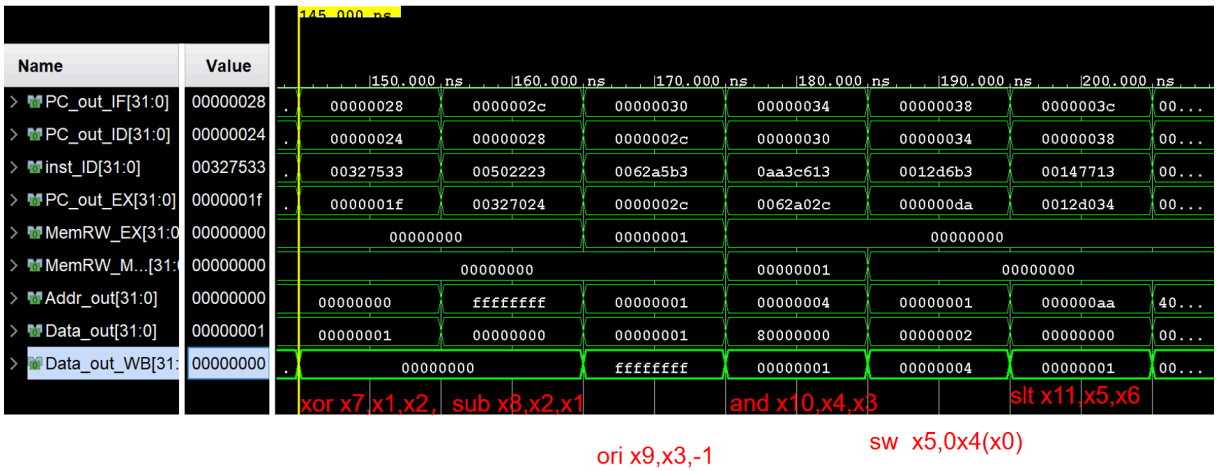
四、 实验结果与分析

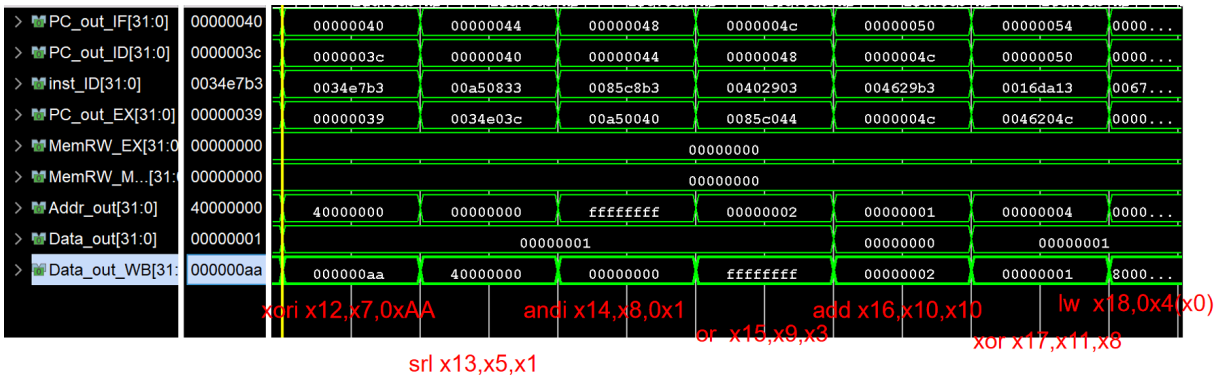
4.1. 五级流水线的搭建与集成

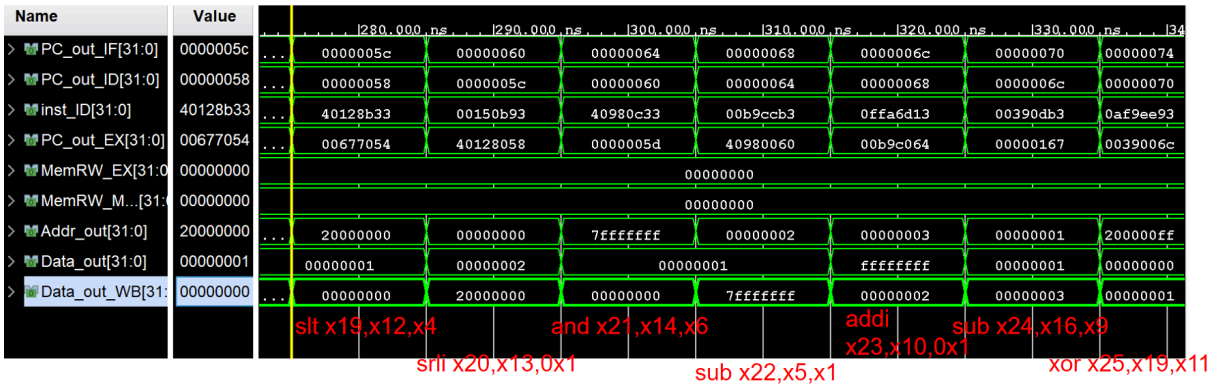
4.1.1. noHazard

仿真结果及分析如下图，符合 noHarzard.S 中的指令


Name
Value
PC_out_IF[31:0]
PC_out_ID[31:0]
inst_ID[31:0]
PC_out_EX[31:0]
MemRW_EX[31:0]
MemRW_M...[31:
Addr_out[31:0]
Data_out[31:0]
Data_out_WB[31:
addi x1,x0,0x1 到 addi x4, x0, 1
lw x5, 0x8(x0)
add x6, x1, x1

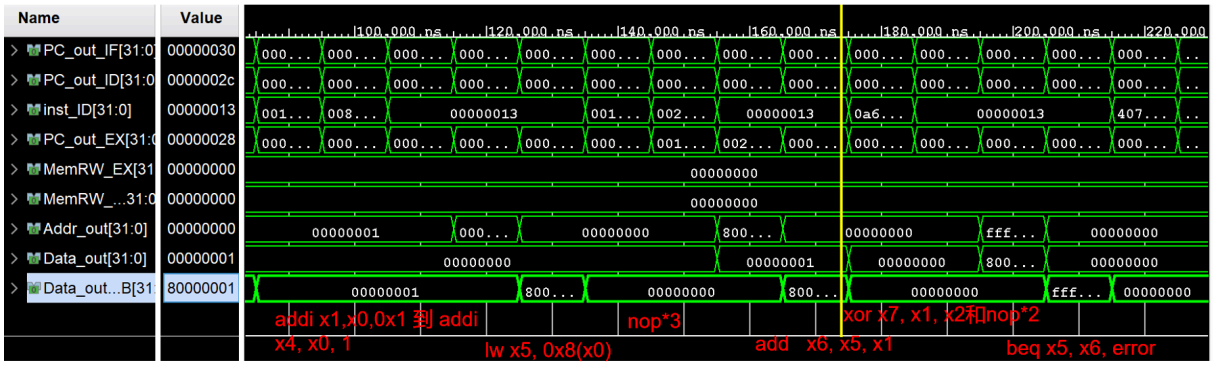

Name
Value
PC_out_IF[31:0]
PC_out_ID[31:0]
inst_ID[31:0]
PC_out_EX[31:0]
MemRW_EX[31:0]
MemRW_M...[31:
Addr_out[31:0]
Data_out[31:0]
Data_out_WB[31:
xor x7,x1,x2,
sub x8,x2,x1
and x10,x4,x3
slt x11,x5,x6
ori x9,x3,-1
sw x5,0x4(x0)

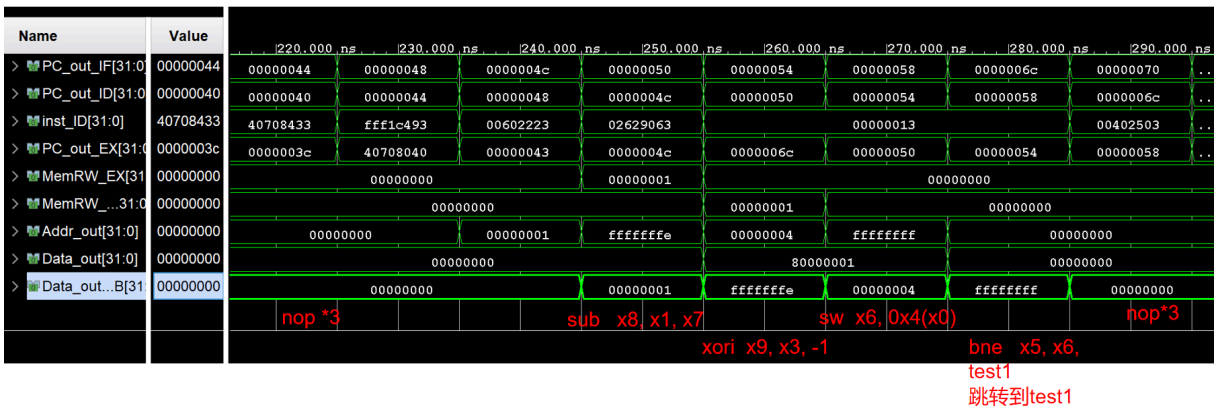

PC_out_IF[31:0]
PC_out_ID[31:0]
inst_ID[31:0]
PC_out_EX[31:0]
MemRW_EX[31:0]
MemRW_M...[31:
Addr_out[31:0]
Data_out[31:0]
Data_out_WB[31:
xori x12,x7,0xAA
andi x14,x8,0x1
add x16,x10,x10
lw x18,0x4(x0)
or x15,x9,x3
xor x17,x11,x8
srl x13,x5,x1

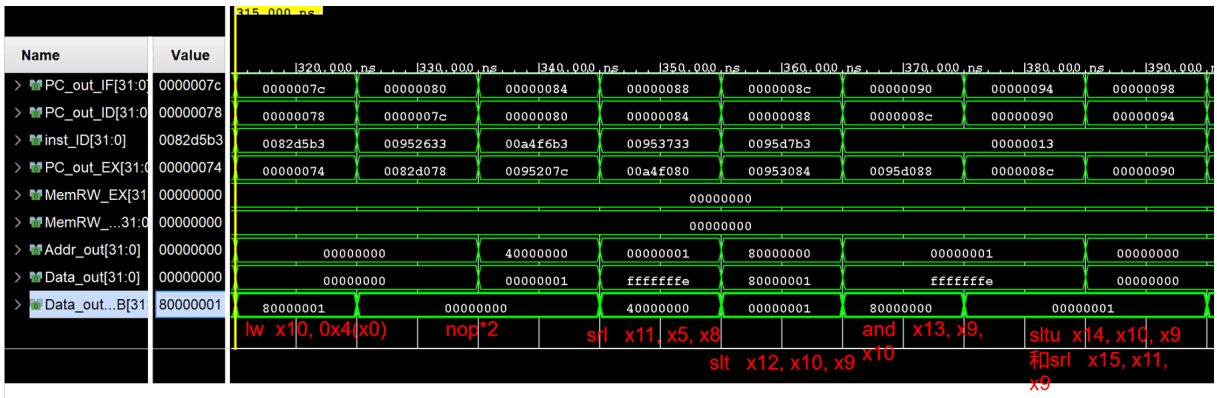

Name
Value
PC_out_IF[31:0]
PC_out_ID[31:0]
inst_ID[31:0]
PC_out_EX[31:0]
MemRW_EX[31:0]
MemRW_M...[31:
Addr_out[31:0]
Data_out[31:0]
Data_out_WB[31:
slt x19,x12,x4
and x21,x14,x6
addi x23,x10,0x1
sub x24,x16,x9
srli x20,x13,0x1
sub x22,x5,x1
xor x25,x19,x11

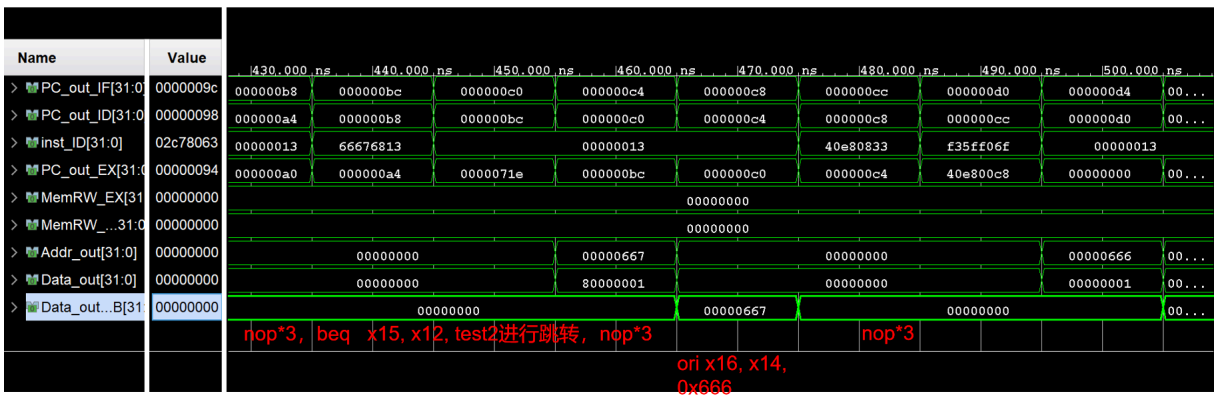
4.1.2. Harzard_NOP

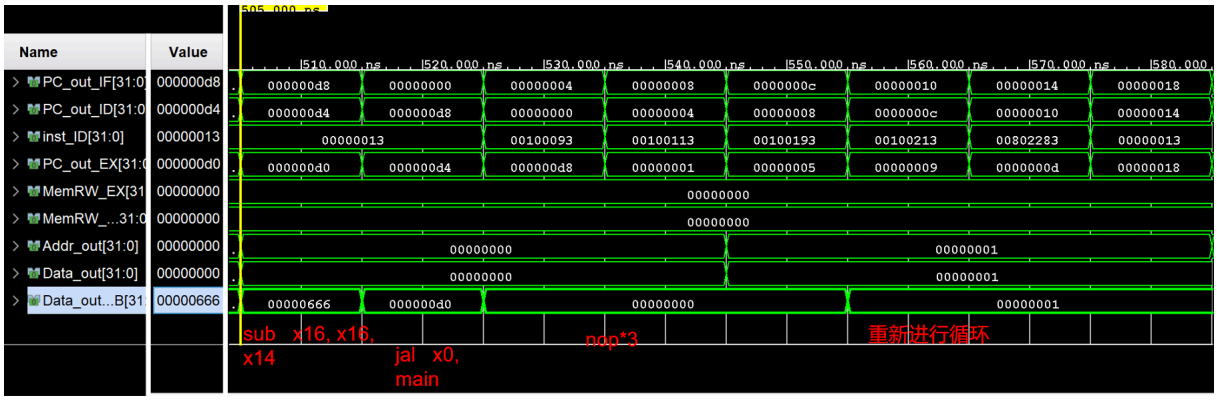
仿真结果及分析如下图，符合 HarzardNOP.S 中的指令











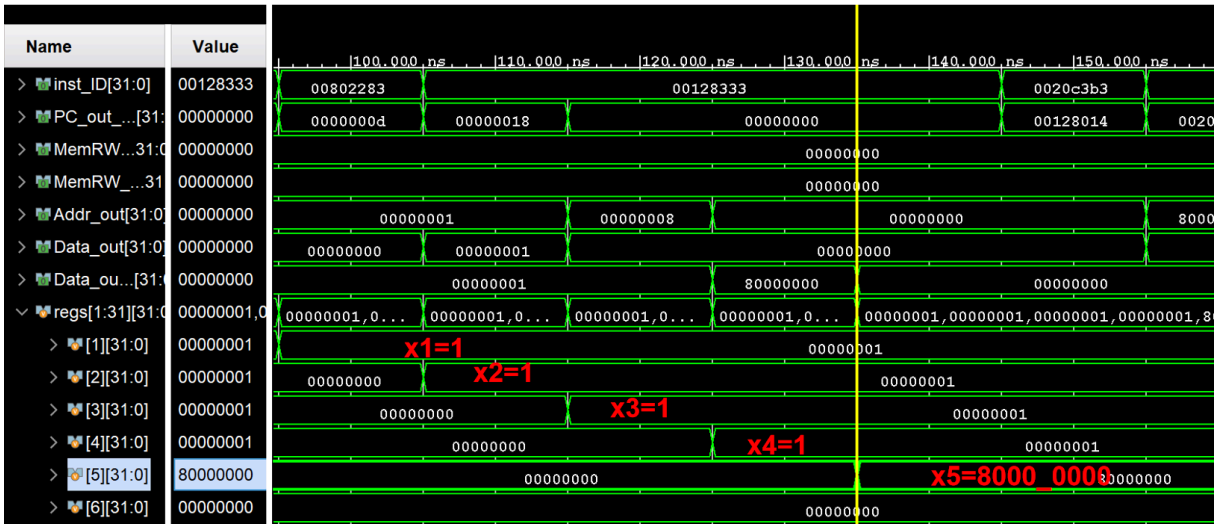
4.2. stall 机制处理流水线冒险

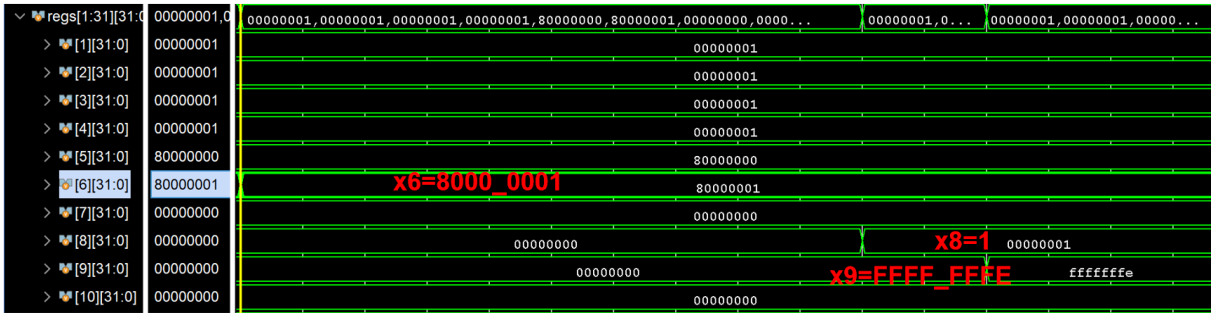
仿真波形与预期相符

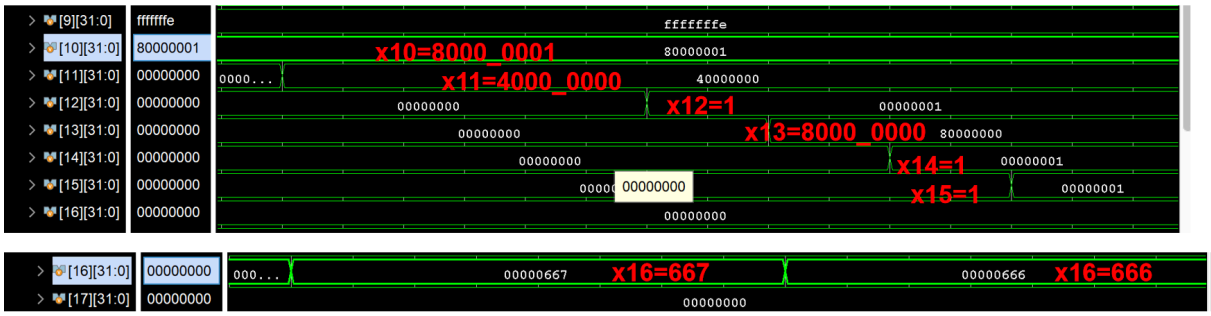
预期各寄存器值如下

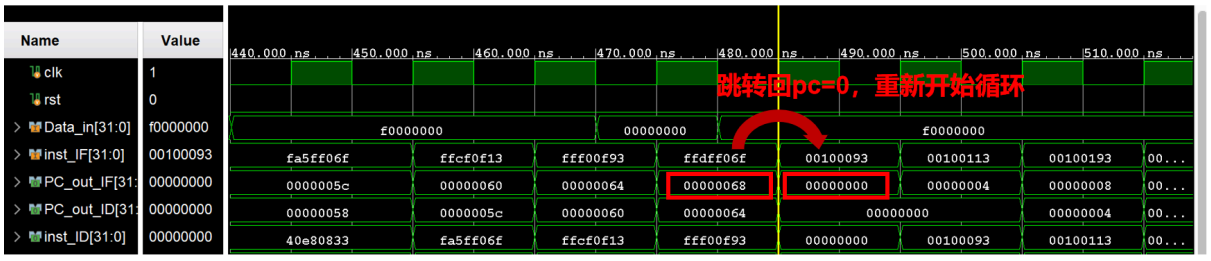
```
main:
    addi  x1, x0, 1      # x1 = 0x1
    addi  x2, x0, 1      # x2 = 0x1
    addi  x3, x0, 1      # x3 = 0x1
    addi  x4, x0, 1      # x4 = 0x1
    lw    x5, 0x8(x0)    # x5 = 0x8000_0000
    add   x6, x5, x1     # x6 = 0x8000_0001
    xor   x7, x1, x2     # x7 = 0
    beq   x5, x6, error  # 不跳转
    sub   x8, x1, x7     # x8 = 1
    xori  x9, x3, -1     # x9 = 0xFFFF_FFFE
    sw    x6, 0x4(x0)    # mem[1] = 0x8000_0001
    bne   x5, x6, test1  # jump to test1
    jal   x30, error     # x30 = pc + 4 则出错

test1:
    lw    x10, 0x4(x0)    # x10 = 0x8000_0001
    srl   x11, x5, x8     # x11 = 0x4000_0000
    slt   x12, x10, x9    # x12 = 0x1
    and   x13, x9, x10    # x13 = 0x8000_0000
    sltu  x14, x10, x9    # x14 = 0x1
    srl   x15, x11, x9    # x15 = 0x1
    beq   x15, x12, test2
    jal   x30, error      # x30 = pc + 4 则出错

test2:
    ori   x16, x14, 0x666 # x16 = 0x0000_0667
    sub   x16, x16, x14   # x16 = 0x0000_0666
    jal   x0, main

error:
    addi  x30, x30, -4    # 定位出错点
loop:
    addi  x31, x0, -1
    jal   x0, loop
```

仿真波形如下









五、 实验讨论、心得

本次实验完成了基于 RISC-V 指令集的五级流水线 CPU 设计，重点实现了冒险检测（Hazard Detection）与停顿处理（Stall）机制。在实验过程中，我深刻体会到了流水线“时序”的重要性。最难忘的调试经历有两点：

数据冒险处理：在解决 Load-Use 冒险时，我发现单纯的流水线停顿会导致 ID 阶段读取到旧数据（0），导致 add 计算错误。通过修改寄存器堆（Register File），实现了内部前递，即在写使能有效且读写地址相同时直接输出写入数据，成功解决了时序冲突。

控制冒险处理：在处理分支跳转时，最初只刷新了 IF 和 ID 阶段，导致跳转指令后紧跟的指令误进入 EX 阶段并执行，造成程序跑飞。后来通过在 EX/MEM 寄存器增加 Flush 逻辑，确保了跳转发生时彻底清除错误路径的指令。 通过观察仿真波形的信号变化，我最终验证了 CPU 能正确处理各种数据依赖和跳转逻辑，深刻理解了硬件并行处理的复杂性与魅力。